FastAPI Webバックエンド
実習指示書

PostgreSQLから本番デプロイまで

目的：HTTP → API → Python → DB → テスト → Docker → Deployment を、1本の実装として理解する

題材：Job Application Tracker API（就活応募管理API）

学習用ガイド / 日本語版

目次

0. 最終目標と完成イメージ
1. 使用技術と、この周回で扱わないもの
2. AI・チュートリアルの使い方
Phase 1 HTTPを理解する
Phase 2 FastAPIでHTTP APIを作る
Phase 3 Request / Response と Pydantic
Phase 4 HTTPエラー処理
Phase 5 PostgreSQL
Phase 6 SQLAlchemy ORM
Phase 7 Alembic
Phase 8 アプリを分割する
Phase 9 Environment Variables
Phase 10 2つ目のtableと関係
Phase 11 Filtering / Pagination
Phase 12 Testing
Phase 13-14 Docker / Compose
Phase 15 CORS
Phase 16-17 本番Deploymentと確認
Phase 18 README
最終試験・読む/読まない一覧・実装順序

0. この実習の最終目標

作るものは Job Application Tracker API（就活応募管理API）。FastAPIに詳しくなること自体ではなく、HTTPリクエストがPostgreSQLに保存され、JSONレスポンスになる流れを説明できるようになることが目標です。

```
Browser / curl
↓ HTTP Request
FastAPI Router
↓
Pydantic Schema
↓
Service
↓
SQLAlchemy Session
↓ SQL
PostgreSQL
↓
ORM Object → Pydantic → JSON HTTP Response
```

公開までの流れも完成させます。

```
GitHub → Docker Image → Render Web Service
↓
Render PostgreSQL
↓
Public HTTPS API
```

☐ メモリ上のCRUD APIを設計・実装できる

☐ PostgreSQLをSQLAlchemy経由で使える

☐ migration、テスト、Docker、公開までを自分で説明できる

1. 使用技術を固定する

```
Python / FastAPI / Pydantic / SQLAlchemy 2.x / psycopg 3
PostgreSQL / Alembic / pytest / HTTPX・FastAPI TestClient
Docker / Docker Compose / Render / Git・GitHub
```

DBアクセスはまず 同期SQLAlchemy を使います。接続URLの概念は `postgresql+psycopg://user:password@host:5432/database` です。

この1周目では次を扱いません。基本を一周してから必要に応じて戻ります。

- async SQLAlchemy / asyncpg
- Redis / Celery / WebSocket / Kubernetes / Microservices
- GraphQL / OAuth / 本格的なReact
- Repository Pattern、DDD、Clean Architectureの厳密な実装

2. AI・チュートリアルの使い方

```
1. 読む → 2. サンプルを動かす → 3. サンプルを閉じる
→ 4. 自分のアプリに自力で実装する → 5. 動作確認する → 6. 説明する
```

特に重要なのは4です。ライブラリ初期化、Dockerfileの定型部分、Alembic設定、pytest fixtureの定型は参考にして構いません。一方で、以下は必ず自分で決めます。

- URL設計、HTTP method、status code

- Pydantic schema、DB schema、制約、SQL query
- エラー処理、CRUD処理、テストケース

> エラー時は、(1)やりたかったこと (2)期待結果 (3)実結果 (4)エラーメッセージ
> (5)自分の仮説、を先に書いてからAIへ相談する。「CRUD全部書いて」は禁止。

Phase 1 — HTTPを理解する

FastAPIを触る前に、request / response / method / path / query parameter / header / body / status code / JSON を区別できるようにします。MDNのHTTP Overviewを読み、以下の意味を調べます。

```
GET / POST / PATCH / DELETE
200 OK / 201 Created / 204 No Content / 400 Bad Request
404 Not Found / 409 Conflict / 422 Unprocessable Content / 500 Internal Server Error
```

課題1：HTTPリクエストを分解する

```
GET /applications?status=interview&limit=20 HTTP/1.1
Host: localhost:8000
Accept: application/json
```

☐ method

☐ path

☐ query parameters

☐ headers

☐ body

課題2：コードを書かずにAPIを設計する

「応募を作る・一覧を見る・1件を見る・変更する・削除する」について、method、URL、成功時のstatus code、存在しない場合のstatus codeを決めます。考えた後に模範的な方向性と照合します。

```
POST /applications
GET /applications
GET /applications/{id}
PATCH /applications/{id}
DELETE /applications/{id}
```

Phase 2 — FastAPIでHTTP APIを作る

2.1 First Steps

FastAPI Tutorialの First Steps を読みます。ここは写経してよい範囲です。最小の `GET /` を作り、`{"message": "job tracker api"}` を返します。次にサンプルを見ず、`GET /health` を追加し、`{"status": "ok"}` を返します。

☐ `@app.get("/health")` は何をするか

☐ app、GET、localhost:8000、8000番ポートとは何か

☐ dictがなぜJSONになるか、/docsを誰が生成するか

2.2 Path Parameters

Tutorialの Path Parameters を読みます。仮データを使って、サンプルを閉じた後に `GET /applications/{application_id}` を自力で作ります。IDは整数にし、存在しないIDへの応答を決めます。

```
applications = {
1: {"company": "Example Inc", "role": "Backend Engineer"},
2: {"company": "Foo Corp", "role": "Software Engineer"},
```

```
}
```

2.3 Query Parameters

Tutorialの Query Parameters を読み、list / dictのままで `GET /applications?status=interview&limit=10` を実装します。limitが0や10000のとき、存在しないstatusのときを自分で決め、validationを入れます。

Phase 3 — Request / Response と Pydantic

3.1 Request Body

Tutorialの Request Body を読み、Itemの例を一度だけ動かします。次に `ApplicationCreate` を自分で設計します。候補は company, role, status, job_url, applied_at, deadline, notes。すべてをoptionalにはせず、必須項目を理由とともに決めます。

status は interested / applied / interview / offer / rejected / withdrawn の6種類。`Literal[...]` またはEnumを使い、任意の文字列を受け取らないようにします。

3.2 POSTを実装

`POST /applications` を実装し、いったんPython listへ追加します。成功は 201 Created。Swaggerの `/docs` で以下を送信し、特に422がいつ発生するか観察します。

☐ 正常データ

☐ companyなし

☐ 不正なstatus

☐ 不正なdeadline

☐ 余計なfield

3.3 Response Model / 3.4 PATCH

Tutorialの Response Model を読み、`ApplicationCreate`、`ApplicationRead`、`ApplicationUpdate` を分けます。DB modelとAPI schemaは同じものではありません。

```
ApplicationCreate: company / role / status / ...
ApplicationRead: id / company / role / status / created_at / ...
ApplicationUpdate: 全フィールドoptional
```

`PATCH /applications/{id}` を実装します。`model_dump(exclude_unset=True)` の意味を調べ、`{"status": "interview"}` だけを送ってもcompanyが消えないことを確認します。

Phase 4 — HTTPエラー処理

Tutorialの Handling Errors を読みます。`HTTPException` はreturnではなくraiseします。存在しないIDに対するGET / PATCH / DELETEはすべて404にします。

> この段階の完成API：GET /health、POST /applications、GET /applications、GET /applications/{id}、PATCH /applications/{id}、DELETE /applications/{id}。ここまではメモリ上で動作すればよい。

Phase 5 — PostgreSQL

ここで初めてDBを接続します。環境差を減らすため、最初はDockerでPostgreSQLだけ起動して構いません。この時点ではDockerを『PostgreSQLを動かす箱』として扱います。

```
DB name: jobtracker
user: jobtracker
password: 開発用password
port: 5432
```

ORMの前にpsqlで `SELECT version();` を実行します。練習用tableでCREATE TABLE、INSERT、SELECT、UPDATE、DELETEを一度だけ直接SQLで実行します。目的は、ORMの裏で最終的にSQLが実行されることの再確認です。

Phase 6 — SQLAlchemy ORM

FastAPI用のSQL tutorialを丸写しせず、SQLAlchemy公式のORM Quick Startで Declare Models / Create an Engine / Create Objects and Persist / Simple SELECT / Make Changes / Deletes を読みます。

6.1 Application ORM model

Userモデルはコピーしません。`class Application(Base)`
を自分で書きます。以下の各列について、型、nullable、unique、index、defaultを決めます。

```
id / company / role / status / job_url / applied_at / deadline
notes / created_at / updated_at
```

6.2 Engine / Session / Dependency Injection

SQLAlchemyのUsing the Sessionを読み、Engine、Connection、Session、Transactionの違いを自分の言葉でノートに書きます。FastAPIのDependenciesを読んで `get_db()` を作り、DB Sessionをrouteへ渡します。

```
HTTP Request → FastAPI → get_db → Session → PostgreSQL
```

6.3 メモリ上のデータを完全に消す

`applications = []` のような一時データを消し、POST→INSERT、GET→SELECT、PATCH→UPDATE、DELETE→DELETEに置き換えます。ORM queryも2.x styleの `select()`
を使います。サーバーを停止・再起動しても応募データが残ることを確認します。

Phase 7 — Alembic

以後、本番アプリのschema管理に `Base.metadata.create_all()` を使いません。Alembic Tutorialを読み、`alembic init`、target_metadata、revision、upgrade、downgrade、headの意味を理解します。

演習としてApplicationに salary_min と salary_max を追加します。PostgreSQLへ直接ALTER TABLEはしません。現在のschema → migration → 新schemaを説明し、migrationを作って `alembic upgrade head` を実行します。DB側にもcolumnがあることを確認します。

Phase 8 — アプリを分割する

この段階でFastAPIのBigger Applicationsを読み、APIRouterで構造化します。main.pyが大きくなってからで構いません。

```
app/
main.py
api/routes/applications.py
core/config.py
db/session.py
models/application.py
schemas/application.py
services/application_service.py
tests/
alembic/ alembic.ini Dockerfile compose.yaml pyproject.toml README.md
```

- routes：path / query / status code / request / responseなどHTTP
- schemas：API入出力とvalidation
- models：DB tableとSQLAlchemy ORM
- services：検索・作成・更新・削除・business logic
- db：Engine / Session / DB接続
- core：設定とenvironment variables

Phase 9 — Environment Variables

DB passwordをコードに書きません。DATABASE_URLとして外から渡します。.env、password、秘密鍵をGit Hubへcommitしないことををgitignoreで確認します。

Phase 10 — 2つ目のtableを追加する

company名を毎回文字列で保存する代わりに、companiesとapplicationsへ分けます。SQLAlchemy Relationship Configurationを読み、Alembic migrationでschemaを変更します。

```
Company: id / name / website
Application: id / company_id / role / ...
```

☐ なぜcompany tableに分けるのか

☐ Primary Key、Foreign Key、1:Nとは何か

☐ 会社をDELETEするときapplicationをどうするか

Phase 11 — Filtering / Pagination

`GET /applications` にstatus、company_id、limit、offsetを追加します。例：`/applications?status=interview&limit=20&offset=0`。SQLAlchemyが生成するSELECT / WHERE / LIMIT / OFFSETを確認し、ORMを魔法として扱わないようにします。

Phase 12 — Testing

FastAPIのTestingを読みます。まず公式サンプルだけを動かした後、自分のAPIへpytest + TestClientのテストを書きます。

必須テスト

☐ health endpoint

☐ 応募作成成功・必須fieldなし・不正なstatus

☐ 応募取得成功・存在しないID

☐ 一覧取得・status filter・pagination

☐ 応募更新・存在しない応募の更新

☐ 応募削除・存在しない応募の削除

DBテスト

production databaseをテストで使いません。jobtracker_testのような別DBを作り、DATABASE_URLとTEST_DATABASE_URLを分けます。FastAPIのdependency overrideを使い、本番SessionをテストSessionへ差し替えます。

```
本番Session
↓ override
テストSession
```

assert response.status_code == 201 と書くなら、「なぜ200ではなく201か」を説明できること。

Phase 13 — Dockerを正式に学ぶ

Docker公式Python guideを参考に、Image / Container / Dockerfile / Build / Run / Port / Volume / Network / Composeを理解します。

13.1 FastAPIをcontainer化

Dockerfileを書きます。定型部分は公式例を参考にして構いませんが、FROM、WORKDIR、COPY、RUN、CMDの1行ずつを説明できることが条件です。

```
FROM ...
WORKDIR ...
COPY ...
RUN ...
CMD ...
```

13.2 Compose

app containerとpostgres containerをComposeで起動します。`docker compose up --build`だけでFastAPI + PostgreSQLが立ち上がる状態にします。

container内ではlocalhost:5432がDBを指さない場合があります。理由を調べ、Compose network上ではdbのようなservice nameをhostnameとして使うことを理解します。

Phase 14 — Docker状態でテスト

`docker compose down` → `docker compose up --build` の後にもAPIが動くことを確認します。POST → DB保存 → app container restart → GETでもデータが残ることを確認し、container filesystemとPostgreSQL volumeの違いを説明します。

Phase 15 — CORS

フロントエンドを付ける段階でFastAPIのCORSを読みます。originはprotocol / domain / portの組み合わせです。http://localhost:3000とhttp://localhost:8000は別originです。

☐ なぜブラウザだけCORSで問題になるか

☐ なぜcurlでは問題にならないか

☐ preflightとは何か

☐ allow_origins=["*"]で終わらせない理由

Phase 16 — 本番Deployment

Renderを使います。GitHubへREADME、source、tests、Dockerfile、compose.yaml、alembicをpushします。.env、DB password、secretはpushしません。

16.1 Render PostgreSQL

RenderでPostgreSQLを作り、Web Serviceから同一regionのDBへつなぐときはInternal URLを使います。SQLAlchemy + psycopg 3向けの `postgresql+psycopg://` dialectになっているか確認します。

16.2 Render Web Service / Migration

Docker runtimeを選び、Web serverを0.0.0.0へbindします。Render側にDATABASE_URLを設定し、passwordをimageへ焼き込みません。本番DBには `alembic upgrade head` を実行します。productionでcreate_allに逃げません。

Phase 17 — 本番確認

☐ GET /health

☐ POST /applications

☐ GET /applications

☐ GET /applications/{id}

☐ PATCH /applications/{id}

☐ DELETE /applications/{id}

本番URLでSwagger /docsまたはcurlを使い、上のすべてを確認します。

Phase 18 — README

READMEには概要、architecture、使用技術、local setup、environment variables、migration方法、test方法、Docker起動方法、API endpoints、deployment URLを書きます。

```
Client
| HTTP/JSON
v
FastAPI → Service → SQLAlchemy ORM → PostgreSQL
```

最終試験

コードを見ずに、次の問いに8〜9割答えられれば実習は成功です。

HTTP GETとPOST、PUTとPATCH、201と200、404、422、request body、query parameter

FastAPI @app.get、Pydantic、response_model、Depends、APIRouter

Database Primary Key、Foreign Key、transaction、commit、rollback、index、ORM、Session、Engine

PostgreSQL 再起動後もデータが残る理由、server、connection

Alembic migration、revision、upgrade head、create_allだけでは不十分な理由

Test unit / integration test、production DBを使わない理由、dependency override

Docker Image / Container、Dockerfile、Compose、port mapping、volume、container内のlocalhost

Deployment ローカルと本番、DATABASE_URLを環境変数にする理由、Render→PostgreSQLの通信、HTTPS requestが届くPython関数

FastAPI Tutorialの「読む / 読まない」最終一覧

必須	必要になったときだけ	今は不要
First Steps	Header Parameters	Advanced User Guideの大半
Path Parameters	Cookie Parameters	custom response class
Query Parameters	Form Data	sub applications
Request Body	File Upload	custom OpenAPI
Response Model	Middleware	WebSocket advanced
Handling Errors	Background Tasks	async DB optimization
Dependencies	Security	
Bigger Applications	OAuth2/JWT	
Testing	WebSockets	
CORS		

実装順序

HTTPを理解
↓ FastAPIでメモリ上CRUD
↓ Pydantic validation
↓ HTTP error
↓ PostgreSQL
↓ SQLAlchemy
↓ FastAPI Dependency Injection
↓ DB CRUD
↓ Alembic
↓ 複数ファイル化
↓ relation / foreign key
↓ filter / pagination
↓ pytest
↓ Docker
↓ Docker Compose
↓ CORS
↓ GitHub
↓ Render PostgreSQL
↓ Render deployment
↓ README

> 最重要ルール：「この1行を書くと、HTTPのどの情報が、どのPython objectになり、どのSQLになり、PostgreSQLのどこが変わり、最終的にどんなHTTP responseになるのか」を追えるようにする。

FastAPI Tutorialをすべて覚える必要はありません。HTTP → FastAPI → Python → SQLAlchemy → SQL → PostgreSQL の一本の流れを理解することが、この実習の本当の目的です。